YouTube Scraper

Scrape YouTube search results, videos, channels, and playlists at scale. Extract titles, view counts, likes, comments, subtitles, channel info, and more — all delivered as clean, structured JSON.

What does YouTube Scraper do?

YouTube Scraper is an Apify Actor that lets you extract data from YouTube without using the official API (no API key or quotas needed). Just provide search keywords or direct YouTube URLs and get structured video data in seconds.

Key features

- 🔍 **Search scraping** — enter keywords and get video results with full metadata
- 🎬 **Direct URL support** — scrape individual videos, entire channels, or playlists
- 📄 **Subtitle extraction** — download and parse video captions as plain text
- 🔄 **Auto-pagination** — automatically loads more results beyond the first page
- 🎯 **Smart filters** — filter by upload date (hour/day/week/month/year) and type (videos/shorts/streams)
- 🛡️ **Anti-blocking** — uses Apify proxy with automatic rotation
- ⚡ **Fast & lightweight** — uses CheerioCrawler (no browser), keeping costs low

Use cases

- **Market research** — analyze trending content in any niche
- **Competitor analysis** — track competitor video performance and publishing cadence
- **Influencer discovery** — find creators by keyword and compare subscriber counts
- **Content monitoring** — track video metrics over time with scheduled runs
- **SEO research** — extract video titles, descriptions, and hashtags for keyword analysis
- **Academic research** — collect public video metadata for studies
- **Lead generation** — find businesses or creators through their YouTube presence

Input

Field	Type	Default	Description
Search Terms	`string[]`	`[]`	YouTube search keywords (e.g. `["web scraping tutorial", "node.js"]`)
YouTube URLs	`string[]`	`[]`	Direct video, channel, or playlist URLs
Max Results	`number`	`50`	Maximum number of videos to scrape (1–5,000)
Scrape Subtitles	`boolean`	`false`	Extract video subtitles/captions (increases runtime)
Video Type	`enum`	`"all"`	Filter: `"videos"`, `"shorts"`, `"streams"`, or `"all"`

Field	Type	Default	Description
Date Filter	`enum`	`"all"`	Filter: `"hour"`, `"day"`, `"week"`, `"month"`, `"year"`, or `"all"`

> **Note:** You must provide at least one search term **or** one YouTube URL.

Input examples

Search for videos:

```
{
    "searchTerms": ["machine learning tutorial", "python for beginners"],
    "maxResults": 50,
    "videoType": "videos",
    "dateFilter": "month"
}
```

Scrape specific videos and a channel:

```
{
    "youtubeUrls": [
        "https://www.youtube.com/watch?v=dQw4w9WgXcQ",
        "https://www.youtube.com/@Google/videos",
        "https://www.youtube.com/playlist?list=PLRqwX-
V7Uu6ZiZxtDDRCi6uhfTH4FilpH"
    ],
    "maxResults": 100,
    "scrapeSubtitles": true
}
```

Get Shorts from the last week:

```
{
    "searchTerms": ["cooking recipes"],
    "maxResults": 30,
    "videoType": "shorts",
    "dateFilter": "week"
}
```

Output

Each video is pushed to the default Apify Dataset as a JSON record. You can download results in JSON, CSV, Excel, or connect via API.

Output fields

Field	Type	Description
`title`	`string`	Video title
`videoId`	`string`	YouTube video ID
`url`	`string`	Full video URL
`thumbnailUrl`	`string`	Highest resolution thumbnail URL
`viewCount`	`number`	Total view count
`likeCount`	`number`	Like count
`commentsCount`	`number`	Comment count
`duration`	`string`	Video duration (e.g. `"12:34"` or `"1:05:30"`)
`uploadDate`	`string`	Upload date (ISO format or relative)
`channelName`	`string`	Channel display name
`channelUrl`	`string`	Channel URL
`subscriberCount`	`string`	Channel subscriber count (e.g. `"1.2M subscribers"`)
`description`	`string`	Full video description
`hashtags`	`string[]`	Hashtags extracted from description
`subtitles`	`string`	Full transcript text (when enabled)
`scrapedAt`	`string`	ISO timestamp of when the data was scraped

Example output

```
{
    "title": "Learn Web Scraping in 20 Minutes",
    "videoId": "abc123xyz",
    "url": "https://www.youtube.com/watch?v=abc123xyz",
    "thumbnailUrl": "https://i.ytimg.com/vi/abc123xyz/maxresdefault.jpg",
    "viewCount": 152340,
    "likeCount": 4200,
    "commentsCount": 312,
    "duration": "20:15",
    "uploadDate": "2025-12-01",
    "channelName": "Code Academy",
    "channelUrl": "https://www.youtube.com/channel/UC1234567890",
    "subscriberCount": "1.2M subscribers",
    "description": "In this tutorial we cover the basics of web scraping...",
    "hashtags": ["#webscraping", "#nodejs", "#tutorial"],
    "subtitles": "Hi everyone, welcome to this tutorial on web scraping...",
    "scrapedAt": "2026-03-06T12:00:00.000Z"
}
```

Integrations

YouTube Scraper can be connected to many other apps and services through Apify integrations:

- **Google Sheets** — export results directly to a spreadsheet
- **Slack / Discord** — get notifications when a run finishes
- **Zapier / Make** — trigger workflows with scraped data
- **Webhooks** — send results to your own API endpoint
- **GitHub Actions** — schedule scrapes in CI/CD pipelines

You can also access results programmatically via the Apify API:

```
# Get results as JSON
curl "https://api.apify.com/v2/datasets/DATASET_ID/items?token=YOUR_TOKEN"
```

Cost & performance

- **Speed:** ~1 second per video (metadata only), ~2–3 seconds with subtitles
- **Cost:** approximately **$0.50–$1.00 per 1,000 videos** on the Apify Free plan
- **Memory:** 256 MB is sufficient for most runs; use 512 MB for 1,000+ videos

Videos	Subtitles	Estimated time	Estimated cost
50	Off	~1 min	~$0.01
500	Off	~5 min	~$0.05
500	On	~12 min	~$0.10
5,000	Off	~30 min	~$0.40

Tips & best practices

1. **Start small** — test with `maxResults: 5` before running large scrapes
2. **Use filters** — narrowing by `dateFilter` and `videoType` improves relevance
3. **Subtitles add time** — only enable `scrapeSubtitles` when you need transcripts
4. **Schedule runs** — set up Apify Schedules for recurring data collection
5. **Monitor usage** — check the Log tab if a run fails; the Actor logs progress for every video

How it works

1. **Search URLs** are built from your keywords with optional protobuf-encoded filters for date and video type
2. **CheerioCrawler** fetches each page and extracts the embedded `ytInitialData` / `ytInitialPlayerResponse` JSON from `<script>` tags — no browser needed
3. Search results are parsed and individual **video page requests** are enqueued (respecting `maxResults`)
4. **Pagination** uses YouTube's innertube continuation API via POST requests with continuation tokens

5. Each video page handler extracts detailed metadata (likes, comments, subscribers, subtitles) and pushes a record to the Apify Dataset

Tech stack

- **Apify SDK v3** — Actor lifecycle, dataset, proxy
- **Crawlee** — `CheerioCrawler` with labeled routing
- **TypeScript 5** — strict mode, ESM
- **Node.js 20**

Project structure

```
├── .actor/
│   ├── actor.json           # Actor metadata & config
│   ├── INPUT_SCHEMA.json    # Apify input schema
│   └── output_schema.json   # Apify output schema
├── src/
│   ├── main.ts              # Entry point — input parsing, crawler setup
│   ├── scraper.ts           # Crawlee router & request handlers
│   ├── types.ts             # TypeScript interfaces & enums
│   └── utils.ts             # JSON extraction, URL helpers, parsers
├── Dockerfile               # Docker image definition
├── package.json             # Dependencies & scripts
├── tsconfig.json            # TypeScript config
└── README.md                # This file
```

Local development

```
# Install dependencies
npm install

# Create a local input file
mkdir -p storage/key_value_stores/default
echo '{ "searchTerms": ["apify tutorial"], "maxResults": 5 }' >
storage/key_value_stores/default/INPUT.json

# Run in dev mode (no build step)
npm run dev

# Or build and run
npm start
```

> **Note:** Apify Proxy is only available on the Apify platform. Local runs execute without proxy and may be rate-limited by YouTube.

Deploying to Apify

Via Git integration (recommended):

1. Push your code to GitHub
2. Go to Apify Console → Actors → Source
3. Set source type to **Git repository** and paste your repo URL
4. Click **Build**

Via CLI:

```
npm install -g apify-cli
apify login
apify push
```

License

ISC